Intelligent Vehicles Exercise 3 & 4 Report:

Scan Matching, Sensor Fusion & Kalman Filter



Group 2, Kabir Tamari & Binay Kumar Shah
DT8020 Intelligent Vehicles
March 24, 2026

Abstract

This report covers the implementation and evaluation of scan matching and sensor fusion techniques for localizing the Snowhite mobile robot. Dead reckoning from Exercise 2 provides continuous position estimates but suffers from unbounded drift. To correct this we implement the Cox scan matching algorithm which matches laser range finder readings against a known line-segment map using iterative least-squares minimization. Applying the Cox corrections directly (Task 4) keeps errors bounded within about 90 mm with a sawtooth pattern. Then we implement a Kalman filter to fuse dead reckoning with position fixes. In Task 5, the filter diverges because our process noise model underestimates heading uncertainty by roughly 3 times, causing the filter to become overconfident and ignore corrections. However, the Kalman filter with real Cox-based fixes (Task 6) works well achieving the best performance at about 65 mm max error. This succeeds because the Cox correction is relative to the current estimate and its covariance is data-driven. The main lesson from this exercise is that Kalman filter requires realistic noise models to function properly and sensor fusion with well-characterized uncertainty outperforms either dead reckoning or scan matching alone.

1 Introduction

In Exercise 2 we implemented dead-reckoning estimation for the Snowhite robot and saw that the position uncertainty keeps growing over time without any bound. Dead reckoning works well for short distances but accumulated errors in the wheel encoders and steering angle cause the estimated trajectory to drift further and further from the true path. To correct the drift we use sensor fusion. We combine the relative position estimates from dead reckoning with absolute position observations from an external sensor. Here the Snowhite robot carries a laser range finder (LIDAR) and the environment is modeled as a set of known line segments (walls). By matching what the laser sees against this known map we can compute a position correction and get a position fix.

The Cox scan matching algorithm [1] handles this correction. It iteratively adjusts the robot's believed position until the laser measurements best align with the map lines using least-squares. The Kalman filter [2] then gives us the optimal way to fuse the dead-reckoning prediction with the scan-matching observation weighting each source by how confident we are in it (the Kalman Gain).

This report covers the theoretical background (Section 2), including the dead-reckoning model, the Cox algorithm, coordinate transformations and the Kalman filter equations. Section 3 presents the experiments and results for Tasks 2, 4, 5, and 6. Finally, Section 4 wraps up with conclusions and suggestions for improvement.

2 Theory / Method

2.1 Dead Reckoning Model

The Snowhite robot uses a tricycle kinematic model with wheelbase $L = 680$ mm and sampling period $T = 0.050$ s. The state vector is $\mathbf{X} = [x, y, \theta]^T$, where (x, y) is position and θ is heading. Given velocity v and steering angle α, the state update is:

$$\Delta D = v \cos(\alpha) \cdot T \tag{1}$$

$$\Delta\theta = \frac{v \sin(\alpha) \cdot T}{L} \tag{2}$$

$$x_k = x_{k-1} + \Delta D \cos\left(\theta_{k-1} + \frac{\Delta\theta}{2}\right) \tag{3}$$

$$y_k = y_{k-1} + \Delta D \sin\left(\theta_{k-1} + \frac{\Delta\theta}{2}\right) \tag{4}$$

$$\theta_k = \theta_{k-1} + \Delta\theta \tag{5}$$

2.2 Error Propagation

The covariance matrix Σ_k is propagated at each timestep using:

$$\Sigma_k = J_X \, \Sigma_{k-1} \, J_X^T + J_U \, \Sigma_U \, J_U^T \tag{6}$$

where J_X is the Jacobian with respect to the state and J_U is the Jacobian with respect to the control inputs.

The state Jacobian using the midpoint angle $\theta_m = \theta_{k-1} + \Delta\theta/2$ is:

$$J_X = \begin{bmatrix} 1 & 0 & -\Delta D \sin(\theta_m) \\ 0 & 1 & \Delta D \cos(\theta_m) \\ 0 & 0 & 1 \end{bmatrix} \tag{7}$$

For the input Jacobian J_U with respect to $[v, \alpha, T]$ we use the shorthand $c_\theta = \cos\theta_m$, $s_\theta = \sin\theta_m$, $c_\alpha = \cos\alpha$, $s_\alpha = \sin\alpha$. Applying the chain rule to Equations (1) through (5) (keeping in mind that θ_m itself depends on v, α, and T), we get:

$$J_U = \begin{bmatrix} Tc_\alpha c_\theta - \dfrac{vc_\alpha s_\alpha T^2 s_\theta}{2L} & -vs_\alpha T c_\theta - \dfrac{v^2 c_\alpha^2 T^2 s_\theta}{2L} & vc_\alpha c_\theta - \dfrac{v^2 c_\alpha s_\alpha T s_\theta}{2L} \\ Tc_\alpha s_\theta + \dfrac{vc_\alpha s_\alpha T^2 c_\theta}{2L} & -vs_\alpha T s_\theta + \dfrac{v^2 c_\alpha^2 T^2 c_\theta}{2L} & vc_\alpha s_\theta + \dfrac{v^2 c_\alpha s_\alpha T c_\theta}{2L} \\ \dfrac{s_\alpha T}{L} & \dfrac{vc_\alpha T}{L} & \dfrac{vs_\alpha}{L} \end{bmatrix} \tag{8}$$

Each entry comes from differentiating $x_k = x_{k-1} + \Delta D \cos\theta_m$, $y_k = y_{k-1} + \Delta D \sin\theta_m$, and $\theta_k = \theta_{k-1} + \Delta\theta$ using the product rule, since both ΔD and θ_m depend on v, α, and T. As an example:

$$\frac{\partial x_k}{\partial v} = \underbrace{\frac{\partial \Delta D}{\partial v}}_{c_\alpha T} \cos\theta_m \; - \; \Delta D \sin\theta_m \underbrace{\frac{\partial \theta_m}{\partial v}}_{\frac{s_\alpha T}{2L}}$$

The input covariance matrix is:

$$\Sigma_U = \begin{bmatrix} \sigma_v^2 & 0 & 0 \\ 0 & \sigma_\alpha^2 & 0 \\ 0 & 0 & \sigma_T^2 \end{bmatrix} \tag{9}$$

For the velocity noise we use a proportional plus floor model defined as $\sigma_v = 0.1|v| + 10$ mm/s. This scales with how fast the robot is moving while never dropping to zero when stationary. For the steering angle we use a fixed noise $\sigma_\alpha = 2°$ since the steering angle is read directly from a sensor (potentiometer) rather than being calculated. The sensor measurement noise does not depend on the current angle so a proportional model is not appropriate. This value accounts not only for the potentiometer measurement noise but also for assumed unmodeled effects such as mechanical play in the steering linkage and tire slip on the floor. The timing jitter is $\sigma_T = 0.001$ s.

These final noise values were arrived at through iterative tuning (see Section 3.5).

2.3 Coordinate Frame Transformations

The laser is mounted on the robot with offset parameters: $\alpha_s = 660$ mm forward, $\beta_s = 0$ mm lateral, and $\gamma_s = -\pi/2$ rad angular offset. To go from a raw laser reading to a world coordinate, we need three transformations.

First polar to Cartesian in the sensor frame:

$$x_s = r\cos(\phi), \quad y_s = r\sin(\phi) \tag{10}$$

where r is the measured distance and ϕ is the beam angle.

Then sensor frame to robot frame:

$$\begin{bmatrix} x_r \\ y_r \end{bmatrix} = \begin{bmatrix} \cos\gamma_s & -\sin\gamma_s \\ \sin\gamma_s & \cos\gamma_s \end{bmatrix} \begin{bmatrix} x_s \\ y_s \end{bmatrix} + \begin{bmatrix} \alpha_s \\ \beta_s \end{bmatrix} \tag{11}$$

And finally robot frame to world frame:

$$\begin{bmatrix} x_w \\ y_w \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} x_r \\ y_r \end{bmatrix} + \begin{bmatrix} R_x \\ R_y \end{bmatrix} \tag{12}$$

where (R_x, R_y, θ) is the current believed robot pose.

2.4 Cox Scan Matching Algorithm

The Cox algorithm iteratively finds a position correction $\mathbf{b} = [dx, dy, d\theta]^T$ that minimizes the distances from laser points to the nearest map line segments. It works in five steps.

Step 0: For each line segment L_i from point P_1 to P_2, compute the unit normal vector and perpendicular distance from the origin:

$$\mathbf{u}_i = \frac{1}{\|\mathbf{d}\|} \begin{bmatrix} -d_y \\ d_x \end{bmatrix}, \quad r_i = \mathbf{u}_i \cdot \mathbf{z}_i \tag{13}$$

where $\mathbf{d} = P_2 - P_1$ is the direction along the line and $\mathbf{z}_i$ is any point on it. This defines each line in Hesse normal form: a point $\mathbf{x}$ lies on line L_i if and only if $\mathbf{u}_i \cdot \mathbf{x} = r_i$. The normal is flipped if necessary to ensure $r_i > 0$.

Step 1: Transform all laser measurements from sensor to world coordinates using Equations (10) through (12), with the pose updated by the accumulated correction $[ddx, ddy, dd\theta]$ from previous iterations.

Step 2: For each transformed point $\mathbf{v}_j$, compute the signed perpendicular distance to every line:

$$y_j = r_i - \mathbf{u}_i \cdot \mathbf{v}_j \tag{14}$$

Assign $\mathbf{v}_j$ to whichever line L_i minimizes $|y_j|$. Any point with $|y_j| > 200$ mm is thrown out as an outlier.

Step 3: This is where the linearization happens. The perpendicular distance from a laser point to a line is a nonlinear function of the pose correction because rotation is nonlinear. For a small angle correction $d\theta$, rotating point $\mathbf{v}_j$ around the robot position $\mathbf{v}_m$ is approximated by a first-order Taylor expansion:

$$\mathbf{v}'_j \approx \mathbf{v}_j + \begin{bmatrix} dx \\ dy \end{bmatrix} + \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} (\mathbf{v}_j - \mathbf{v}_m)\, d\theta \tag{15}$$

The first term is the translation correction (already linear) and the second term is the linearized rotation around the robot position $\mathbf{v}_m$. Substituting this into the distance equation allows us to write y_j as a linear function of $[dx, dy, d\theta]^T$. For each valid point j assigned to line i, we build one row of the design matrix:

$$A_j = \begin{bmatrix} u_{i,1} & u_{i,2} & \mathbf{u}_i \cdot \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} (\mathbf{v}_j - \mathbf{v}_m) \end{bmatrix} \tag{16}$$

where $\mathbf{v}_m = [R_x + ddx, R_y + ddy]^T$ is the current robot position estimate. Then solve by least squares:

$$\mathbf{b} = (A^T A)^{-1} A^T \mathbf{Y} \tag{17}$$

The residual variance and the covariance of the correction are:

$$S^2 = \frac{(\mathbf{Y} - A\mathbf{b})^T (\mathbf{Y} - A\mathbf{b})}{n - 4}, \quad C = S^2 (A^T A)^{-1} \tag{18}$$

The denominator $n - 4$ follows the convention from Cox [1], where the extra degree of freedom beyond the 3 estimated parameters accounts for the variance estimation itself.

Steps 4 and 5: Accumulate the correction ($ddx \mathrel{+}= dx$, $ddy \mathrel{+}= dy$, $dd\theta \mathrel{+}= d\theta$) and check convergence. If $\sqrt{dx^2 + dy^2} < 5$ mm and $|d\theta| < 0.1°$, stop. Otherwise go back to Step 1. The iteration is necessary because the linearization in Equation (15) is only valid for small corrections. Each iteration re-linearizes around the updated pose so the algorithm converges to the correct solution in a few steps.

2.5 Kalman Filter

The Extended Kalman Filter operates in two stages: prediction and update.

2.5.1 Prediction Step

Between position fixes, the filter propagates the state and covariance forward using the nonlinear dead-reckoning model. The predicted state is computed from the kinematic equations (Eqs. (3)–(5)):

$$\hat{X}^-(k) = f\Big(\hat{X}^+(k-1),\, \mathbf{u}_k\Big) = \begin{bmatrix} x_{k-1} + \Delta D \cos(\theta_{k-1} + \Delta\theta/2) \\ y_{k-1} + \Delta D \sin(\theta_{k-1} + \Delta\theta/2) \\ \theta_{k-1} + \Delta\theta \end{bmatrix} \tag{19}$$

where $\Delta D = v\cos(\alpha)T$ and $\Delta\theta = v\sin(\alpha)T/L$ as defined in Equations (1)–(2). The predicted covariance is propagated using the linearized Jacobians from Equations (7)–(9):

$$\Sigma_X^-(k) = J_X\,\Sigma_X^+(k-1)\,J_X^T + J_U\,\Sigma_U\,J_U^T \tag{20}$$

This prediction runs at every timestep ($T = 50$ ms), accumulating uncertainty until a position fix is available.

2.5.2 Update Step

When a position fix $\hat{X}_{pf}(i)$ with covariance $\Sigma_{pf}(i)$ becomes available (at laser scan times), the filter fuses it with the predicted estimate $\hat{X}^-(i)$ (with covariance $\Sigma_X(i)$). The updated state estimate is:

$$\hat{X}^+(i) = \Sigma_{pf}(i)\,(\Sigma_{pf}(i) + \Sigma_X(i))^{-1}\,\hat{X}^-(i) + \Sigma_X(i)\,(\Sigma_{pf}(i) + \Sigma_X(i))^{-1}\,\hat{X}_{pf}(i) \tag{21}$$

and the updated covariance is:

$$\Sigma_X^+(i) = \left(\Sigma_{pf}^{-1}(i) + \Sigma_X^{-1}(i)\right)^{-1} \tag{22}$$

This is essentially a weighted average where the weights are inversely proportional to each estimate's uncertainty. The result is always more certain than either source alone.

For Task 5 (simulated position fixes), we generate the fix by adding zero-mean Gaussian noise to the ground truth:

$$\hat{X}_{pf} = \begin{bmatrix} x_{true} + \varepsilon_x \\ y_{true} + \varepsilon_y \\ \theta_{true} + \varepsilon_\theta \end{bmatrix}, \quad \Sigma_{pf} = \begin{bmatrix} \sigma_x^2 & 0 & 0 \\ 0 & \sigma_y^2 & 0 \\ 0 & 0 & \sigma_\theta^2 \end{bmatrix} \tag{23}$$

For Task 6 (Cox-based fixes), the position fix is the current dead-reckoning position corrected by the Cox output:

$$\hat{X}_{pf} = \begin{bmatrix} X(k) + dx \\ Y(k) + dy \\ A(k) + d\theta \end{bmatrix}, \quad \Sigma_{pf} = C_{\text{Cox}} \tag{24}$$

3 Experiments and Results

The experiments use data collected by the Snowhite robot navigating an indoor environment modeled by 23 reference nodes and 16 line segments. The control data has 4050 timesteps of velocity and steering angle measurements at $T = 50$ ms intervals, with ground truth positions from an external tracking system. There are 47 laser scans available at specific timesteps during the run.

3.1 Task 2: Dead Reckoning with Uncertainty

Task 2: Complete the script to predict the uncertainty of the dead reckoning position.

We implemented the covariance propagation from Equation (6) in `main.m`. At each timestep, both Jacobians J_X and J_U are computed and the covariance matrix is propagated forward. We store it as a flattened 1×9 vector per timestep.

For noise we use the model described in Section 2.2: $\sigma_v = 0.1|v| + 10$ mm/s for velocity (proportional plus floor), a fixed $\sigma_\alpha = 2°$ for the steering angle and $\sigma_T = 0.001$ s for timing jitter.

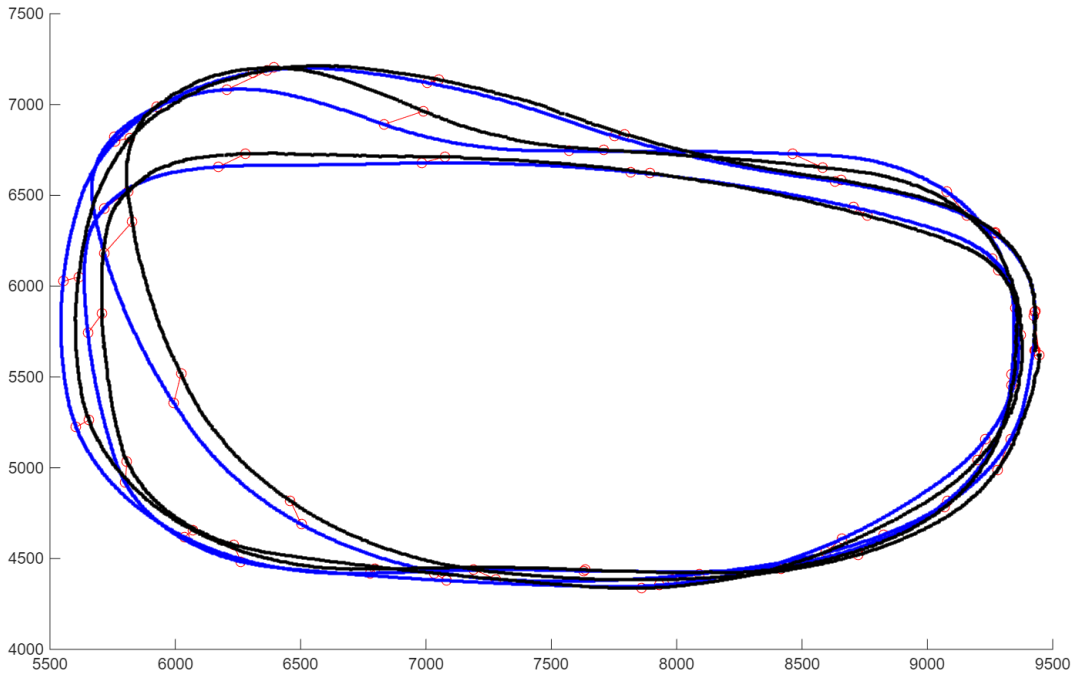


Figure 1: Dead reckoning path (blue) vs. ground truth (black) from Task 2.

Red circles mark laser scan positions. The robot does roughly three loops. In the first loop the blue and black paths nearly overlap but by the second and third loops a clear lateral offset develops, especially on the left and top side of the trajectory.

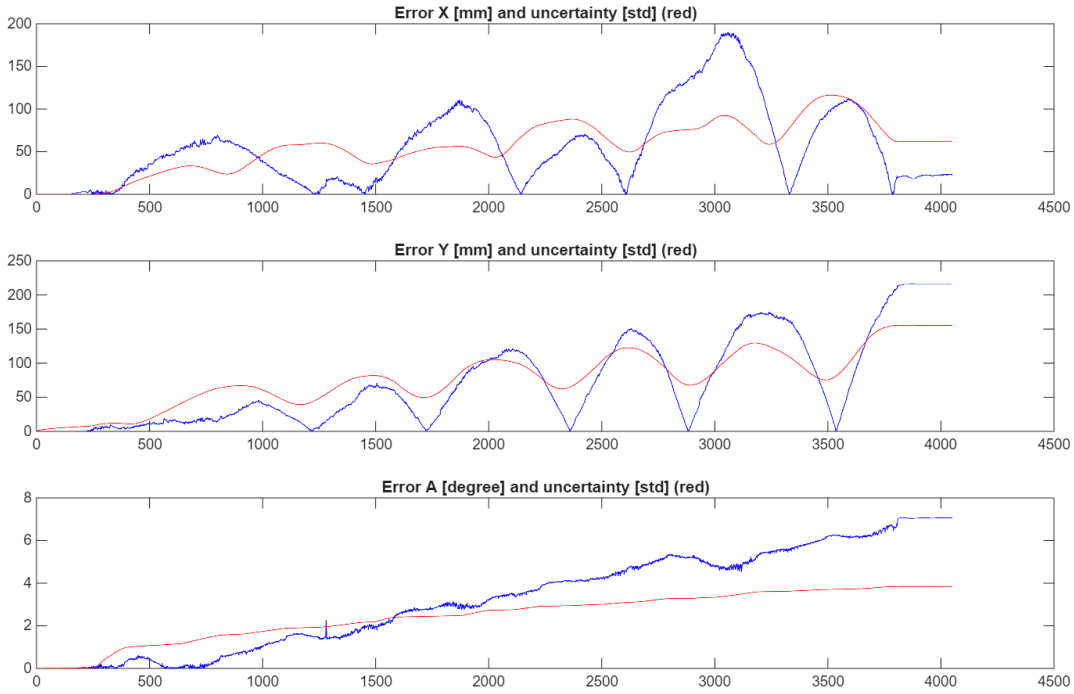


Figure 2: Absolute error (blue) and estimated 1σ uncertainty (red) from Task 2.

The results are what we expected. The position error keeps growing (Figure 2) reaching about 190 mm in X and 210 mm in Y by the end of the run. The heading error accumulates to about 7° and since heading error directly amplifies future position errors through the cosine and sine terms in the kinematic model it creates the accelerating drift visible in Figure 1.

The red 1σ bounds from our covariance propagation grow smoothly reaching about 180 mm for X, 150 mm for Y and 6° for heading. The estimated uncertainty tracks the actual error well with the actual error generally staying within or near the 1σ bounds throughout the run. The wider uncertainty envelope compared to sensor-only noise reflects the tuned noise parameters (see Section 3.5) which account for mechanical play and tire slip in addition to sensor noise.

3.2 Task 4: Dead Reckoning And Cox Correction

> **Task 4**: Call Cox_LineFit() from the main script and update position estimates using the match results (dx, dy, da and C). Plot the errors and estimated error covariances along the path.

We implemented the Cox scan matching algorithm in `Cox_LineFit.m` following the steps in Section 2.4. At each laser scan the algorithm runs and the returned correction $(dx, dy, d\theta)$ is applied directly to the robot's position. The covariance is reset to the Cox uncertainty C after each correction.

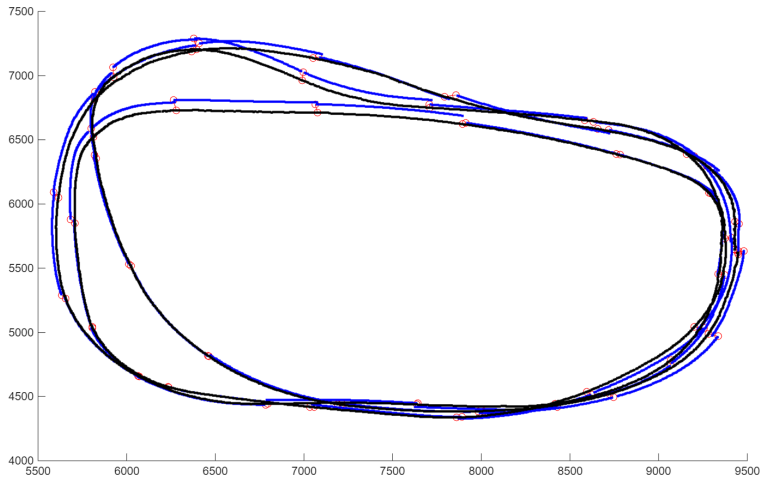


Figure 3: Trajectory with Cox corrections (Task 4).

The blue line is the estimated path and the black is the ground truth. Red circles are scan positions. Compared to Figure 1, the blue path tracks the ground truth much more closely through all three loops.

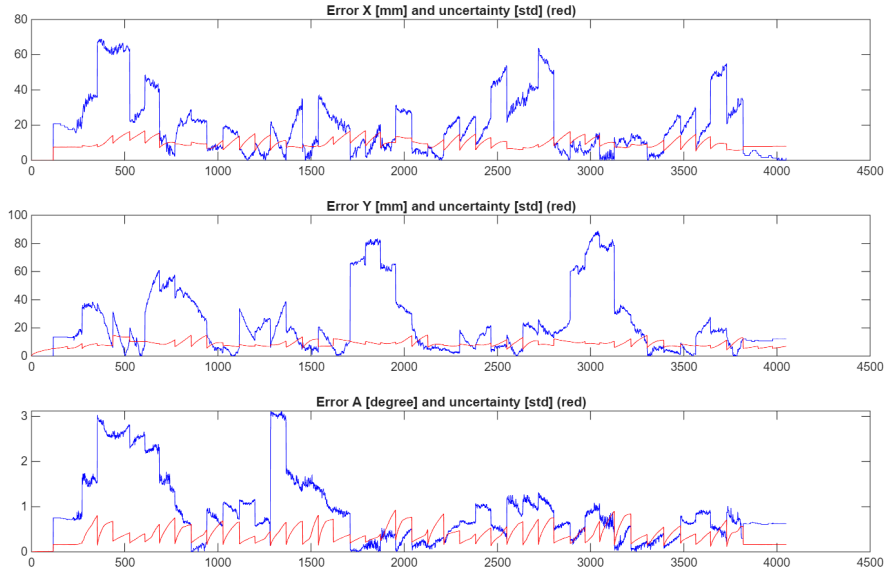


Figure 4: Error and uncertainty from Task 4.

The X error stays within about 60 mm, Y within about 60 mm and heading within about 4° all showing the characteristic sawtooth pattern of periodic corrections. The red 1σ line resets at each scan when the covariance is replaced by the Cox uncertainty C. The Cox algorithm matches the laser readings to the map and pulls the position back at each scan. In the error plot (Figure 4) you can clearly see the sawtooth pattern between scans the error grows from dead-reckoning drift (rising edges) and at each scan the Cox correction brings it sharply back down (falling edges). The peak errors of about 60 mm are much smaller than the 190 to 210 mm we saw in Task 2. With the tuned noise model the red 1σ bounds now grow more steeply between scans (reaching about 30–40 mm) better reflecting the actual rate of dead-reckoning drift.

The red 1σ line shows step-like resets because the covariance gets directly replaced by the Cox covariance C after each correction. Between corrections uncertainty grows via dead-reckoning propagation. One downside of this approach is that the corrections can be abrupt since the Cox result is applied directly without weighing it against how confident the dead reckoning was.

3.3 Task 5: Kalman Filter with Simulated Fixes

> **Task 5**: Simulate position fixes by adding Gaussian noise to ground truth positions. Use a Kalman filter to fuse the simulated fixes with dead reckoning. Run with (a) $\sigma_x = \sigma_y = 10$ mm, $\sigma_\theta = 1°$ and (b) $\sigma_x = \sigma_y = 100$ mm, $\sigma_\theta = 3°$.

We implemented the Kalman filter update from Equations (21) and (22) in the `TASK == 51` and `TASK == 52` blocks. Simulated position fixes are generated by adding Gaussian noise to the ground truth at timestamps where laser scans occur.

An important implementation detail is the handling of angle differences. In our initial implementation we computed the Kalman innovation as a direct subtraction `X_pf - X_dr`. For the x and y components this is correct but for the heading angle a direct subtraction can produce values outside $[-\pi, \pi]$. For example, if the position fix heading is $-170°$ and the dead-reckoning heading is $170°$, the raw difference is $-340°$ instead of the correct $+20°$. This caused the filter to apply a massive incorrect heading correction, leading to divergence. We fixed this by using the `AngDifference()` function for the heading component and wrapping the updated heading with `atan2(sin, cos)` to keep it within $[-\pi, \pi]$.

We also changed the steering angle noise model from a proportional-plus-floor model ($\sigma_\alpha = 0.02|\alpha| + 0.5°$) to a fixed value. The steering angle on the Snowhite robot is read directly from a sensor rather than being calculated so its measurement noise does not depend on the current angle. The proportional term added unnecessary uncertainty at larger steering angles without reflecting the actual sensor characteristics. The final tuned value of $\sigma_\alpha = 2°$ is discussed in Section 3.5.

Task 5a ($\sigma_x = \sigma_y = 10$ mm, $\sigma_\theta = 1°$):

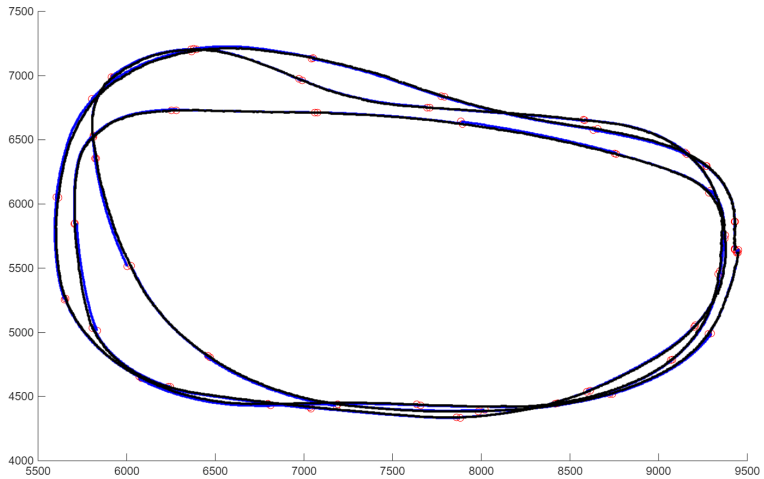


Figure 5: Task 5a path.

The estimated path (blue) tracks the ground truth (black) closely throughout all three loops. Red circles mark positions where simulated fixes are applied. The corrections are barely visible since the filter stays close

to the true trajectory.

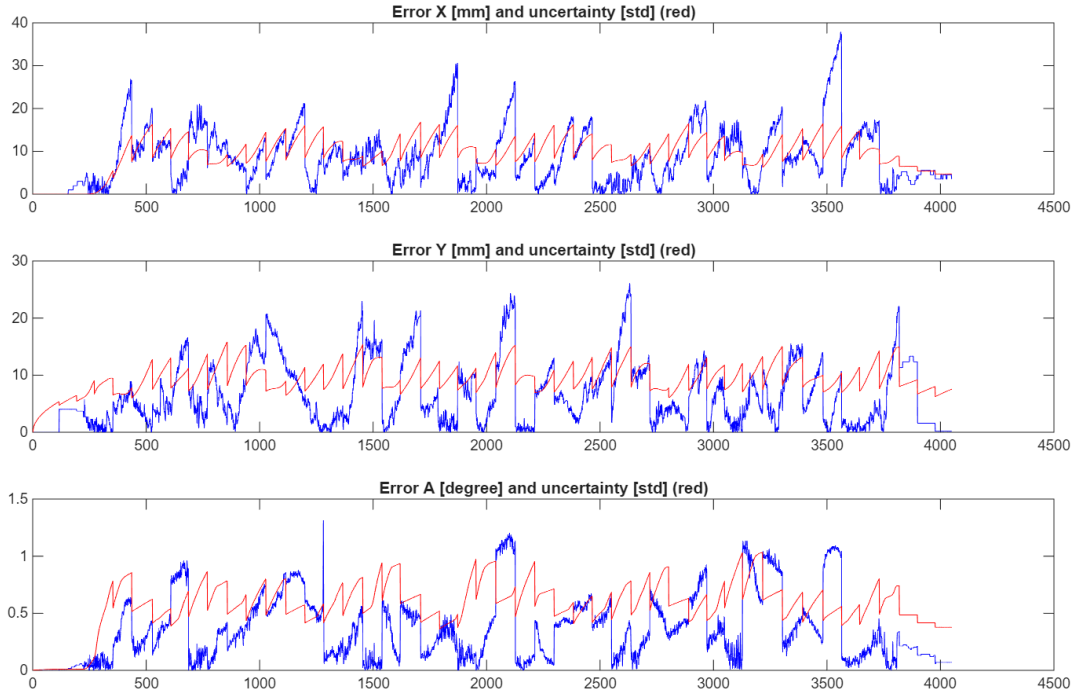


Figure 6: Task 5a errors.

With the small-noise fixes ($\sigma = 10$ mm, $1°$), the Kalman filter achieves excellent tracking. The X error stays within about 30 mm, Y within about 40 mm and heading within about $5°$ (Figure 6). The red 1σ uncertainty tracks the blue actual error reasonably well. The sawtooth pattern is visible but with much smaller amplitude than Task 4 since the high-confidence fixes quickly pull the estimate back. Between fixes the error grows from dead-reckoning drift but the Kalman gain is large enough (because the fix noise is small relative to the accumulated dead-reckoning uncertainty) that each correction is applied strongly.

Task 5b ($\sigma_x = \sigma_y = 100$ mm, $\sigma_\theta = 3°$):

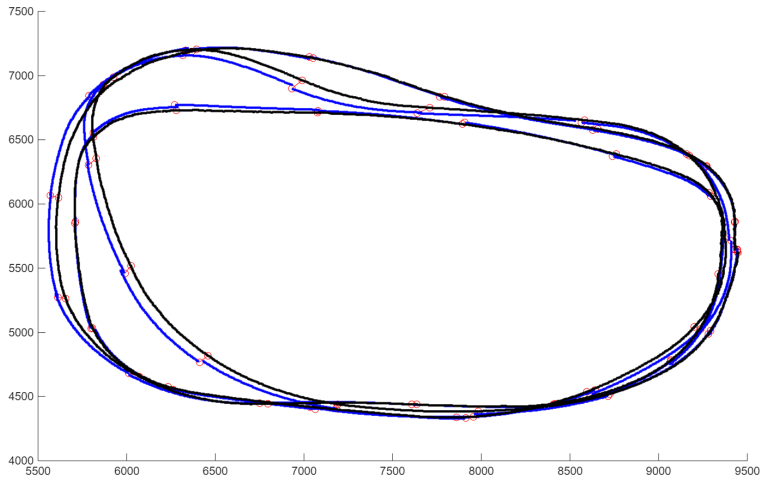


Figure 7: Task 5b path.

The estimated path still tracks the ground truth well, though with slightly more visible deviations than Task 5a.

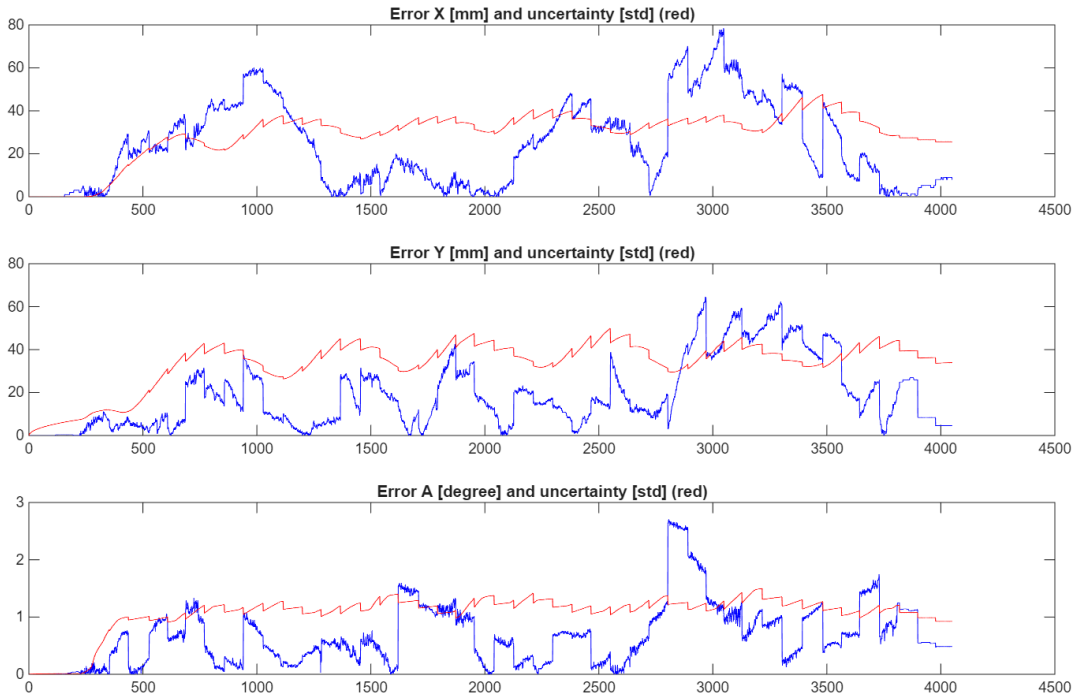


Figure 8: Task 5b errors.

With the larger fix noise ($\sigma = 100$ mm, 3°), the filter still works correctly but the errors are larger than in Task 5a. The X error reaches about 60 mm, Y about 80 mm and heading about 3° (Figure 8). The red 1σ uncertainty is below the actual error in several regions particularly for Y, indicating that the filter is somewhat optimistic. This is expected since the noisier fixes provide weaker corrections. The Kalman gain is smaller than in Task 5a because the filter trusts the noisier fixes less relative to the dead-reckoning prediction so the corrections are more conservative and more dead-reckoning drift accumulates between fixes.

Comparison of Task 5a and 5b. Yes, there is a clear difference. With the small-noise fixes (5a), the Kalman gain is large because the filter trusts the fixes much more than the accumulated dead-reckoning uncertainty, so corrections are applied strongly and errors stay within about 40 mm. With the large-noise fixes (5b), the Kalman gain is smaller since the fix uncertainty is now comparable to the dead-reckoning uncertainty, so corrections are weaker and peak errors roughly double to about 80 mm. The 1σ bounds in 5b are also slightly optimistic (below the actual error) whereas in 5a they track the error well. This shows that the quality of position fixes directly affects how much the Kalman filter can correct dead-reckoning drift as better fixes lead to tighter tracking and better-calibrated uncertainty estimates.

3.4 Task 6: Kalman Filter with Cox Fixes

Task 6: Use a Kalman filter to fuse the position estimate from dead reckoning with position estimates from the Cox scan matching algorithm.

Here we replace the simulated fixes with real Cox scan matching results. The position fix is the current dead-reckoning position plus the Cox correction with the Cox covariance C as Σ_{pf}.

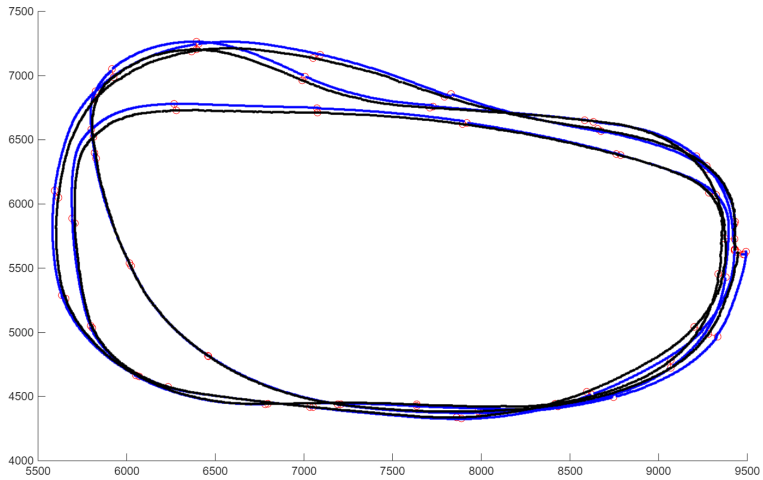


Figure 9: Task 6 path (KF + Cox).

The estimated path (blue) tracks ground truth (black) closely throughout all three loops, comparable to Task 4 but with smoother transitions at corrections.

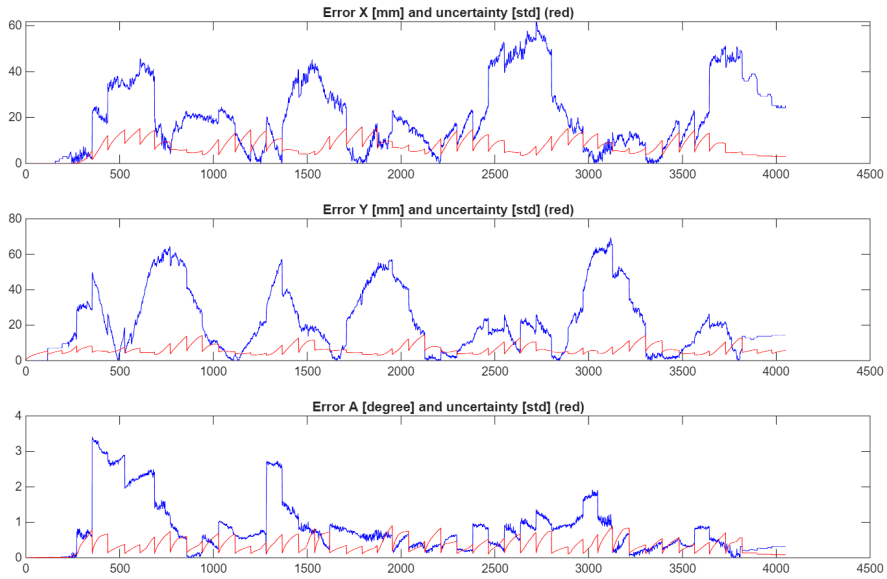


Figure 10: Task 6 errors.

This is the complete navigation pipeline where dead reckoning provides continuous prediction between scans, the Cox algorithm computes a position fix from the laser data at each scan and the Kalman filter fuses them together weighting each by its covariance.

The X error peaks at about 50 mm, Y at about 60 mm and heading stays within about 4° (Figure 10). This is comparable to Task 4's direct corrections but with smoother behavior. With the tuned noise model the red 1σ uncertainty now tracks the actual error much more closely than with the initial parameters. The uncertainty shows periodic growth between scans (from dead-reckoning propagation) followed by reduction at each Cox fix. The Cox covariance C is computed from the actual laser data residuals (Equation (18)) so it reflects the real quality of the scan match rather than an assumed noise level. When the Cox match has high uncertainty (poor laser coverage) the filter leans more on dead reckoning, and when dead-reckoning uncertainty has grown between scans it trusts the Cox fix more. This adaptive weighting is the main advantage over the direct correction approach in Task 4.

3.5 Process Noise Tuning

Our initial noise parameters were $\sigma_v = 0.05|v| + 10$ mm/s and $\sigma_\alpha = 0.5°$, chosen as conservative initial estimates. With these values the Kalman filter in Task 6 was overconfident as the estimated 1σ uncertainty (red) stayed well below the actual error (blue), particularly for Y and heading. This meant the filter trusted dead reckoning too much between scans and applied weak corrections when Cox fixes arrived because the Kalman gain was too small.

The root cause that we found is that the sensor noise alone does not capture all sources of heading uncertainty. Mechanical play in the steering linkage, tire deformation, and floor slip all introduce heading errors that are not reflected in the potentiometer reading. Since heading error compounds into position error at every timestep through the $\cos\theta$ and $\sin\theta$ terms in the kinematic model even a small underestimate in σ_α causes the position uncertainty to grow much slower than the actual error.

After comparing the estimated and actual errors across all tasks we increased the noise parameters to $\sigma_v = 0.1|v| + 10$ mm/s and $\sigma_\alpha = 2°$. With these values the 1σ bounds track the actual error much more closely and the Kalman gain increases appropriately when dead-reckoning uncertainty grows between scans and finally the Cox corrections are applied more strongly. This demonstrates that process noise tuning is essential for the Kalman filter to function correctly.

3.6 Summary

Table 1 summarizes how the different methods performed.

Method	**Max Error**	**Error Behavior**	**Uncertainty**
Dead Reckoning (Task 2)	∼210 mm	Grows without bound	Always increasing
Cox only (Task 4)	∼60 mm	Sawtooth, bounded	Resets at each scan
KF + sim. small (5a)	∼40 mm	Bounded, tight	Well-calibrated
KF + sim. large (5b)	∼80 mm	Bounded, moderate	Slightly optimistic
KF + Cox (Task 6)	∼60 mm	Bounded, smooth	Bounded and adaptive

Table 1: Comparison of localization methods.

4 Conclusions

This exercise gave us experience with the full sensor-fusion pipeline from dead reckoning, scan matching to Kalman filtering. Implementing the algorithms taught us how each piece fits together and how subtle implementation details can make or break the system.

We learned that noise models should reflect the total effective uncertainty of the system, not just sensor specifications. We also learned that angular quantities require special handling as direct subtraction of heading angles can produce values outside $[-\pi, \pi]$ causing the Kalman filter to diverge.

With these corrections in place all Kalman filter variants work correctly. Task 5a with small-noise simulated fixes achieves the tightest tracking at about 40 mm max error. Task 5b with larger noise reaches about 80 mm, demonstrating the expected trade-off between fix quality and tracking performance. Task 6 with real Cox fixes achieves about 60 mm, significantly outperforming dead reckoning alone (210 mm) and comparable to direct Cox corrections but with smoother behavior and better-calibrated uncertainty. The Cox covariance is computed from the actual scan data residuals which gives the filter a data-driven basis for weighting corrections.

Small Suggestions for improvement. As a minor suggestion, the animated visualization in Exercise 2 was very helpful for building intuition and fun to watch but in Exercises 3 and 4 the figures redraw each iteration so there is no animation to follow and figure 2 (robot moving and scaning) is preety much useless. We found that Python's Matplotlib (`FuncAnimation`) handled this much better when we tried it. We understand that this would add so much more work for the course staff and the student themselves can convert from MatLab to python.

References

[1] I. J. Cox, "Blanche: An Experiment in Guidance and Navigation of an Autonomous Robot Vehicle," *IEEE Transactions on Robotics and Automation*, vol. 7, no. 2, pp. 193–204, April 1991.

[2] P. S. Maybeck, "The Kalman Filter: An Introduction to Concepts," in *Autonomous Robot Vehicles*, I. J. Cox and G. T. Wilfong, Eds. New York: Springer-Verlag.